Python
import
Estrutura de Repetição- comando While
while/else
break

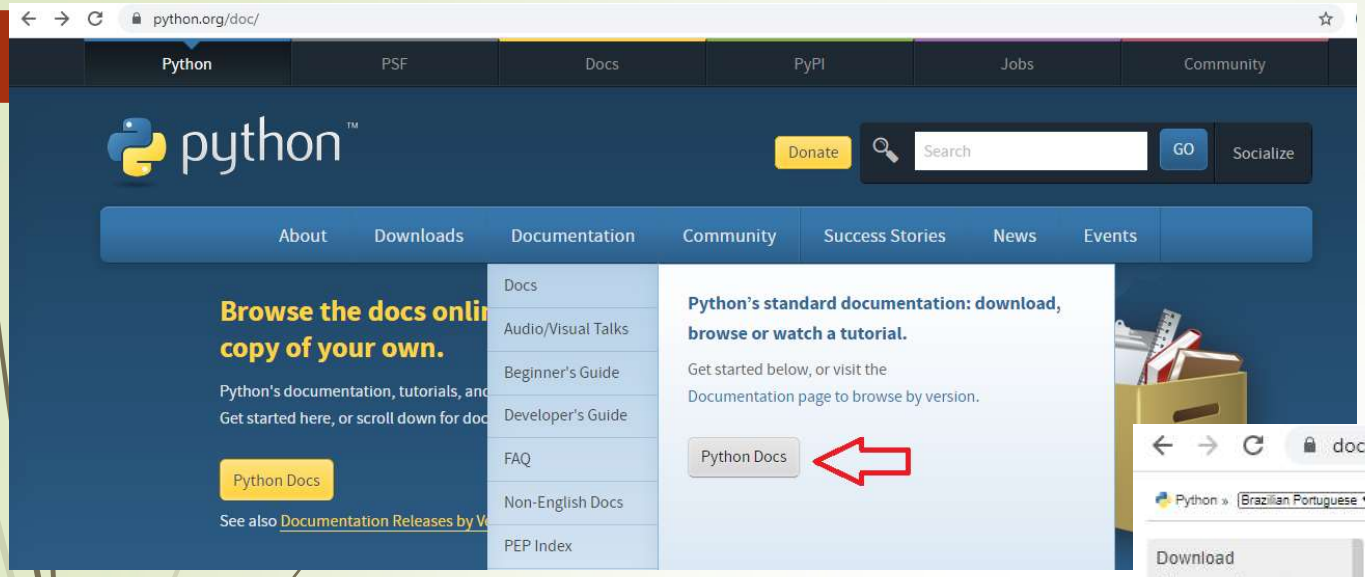
INE 5402 - INE 5603

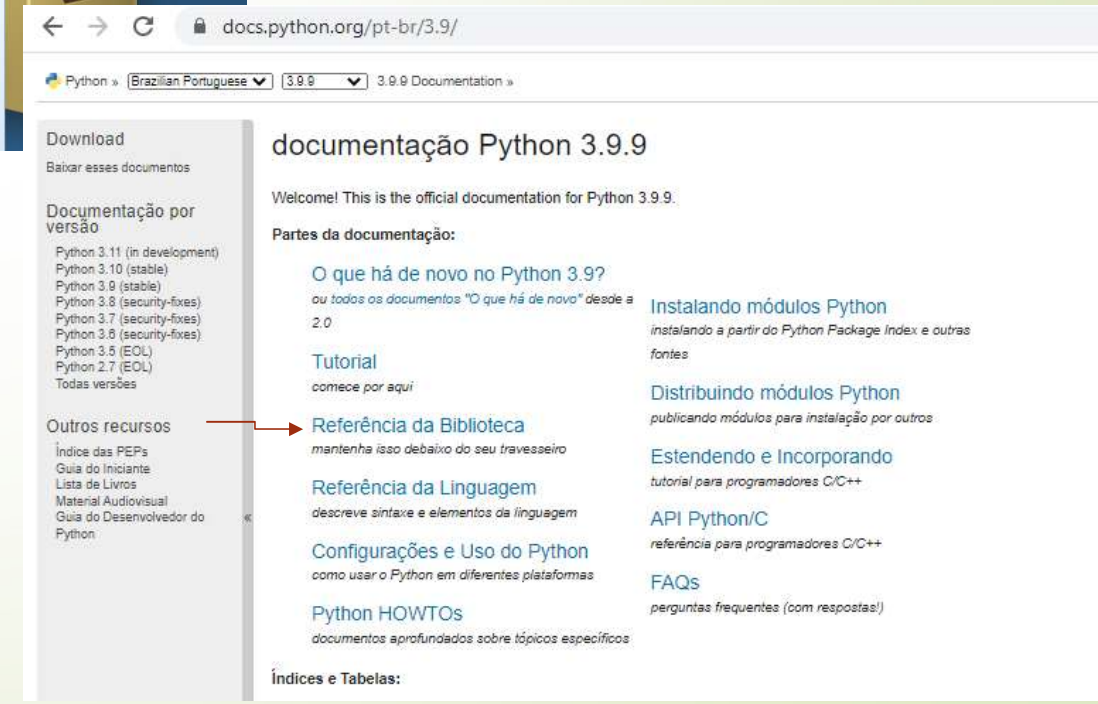
Prof Luciana Rech

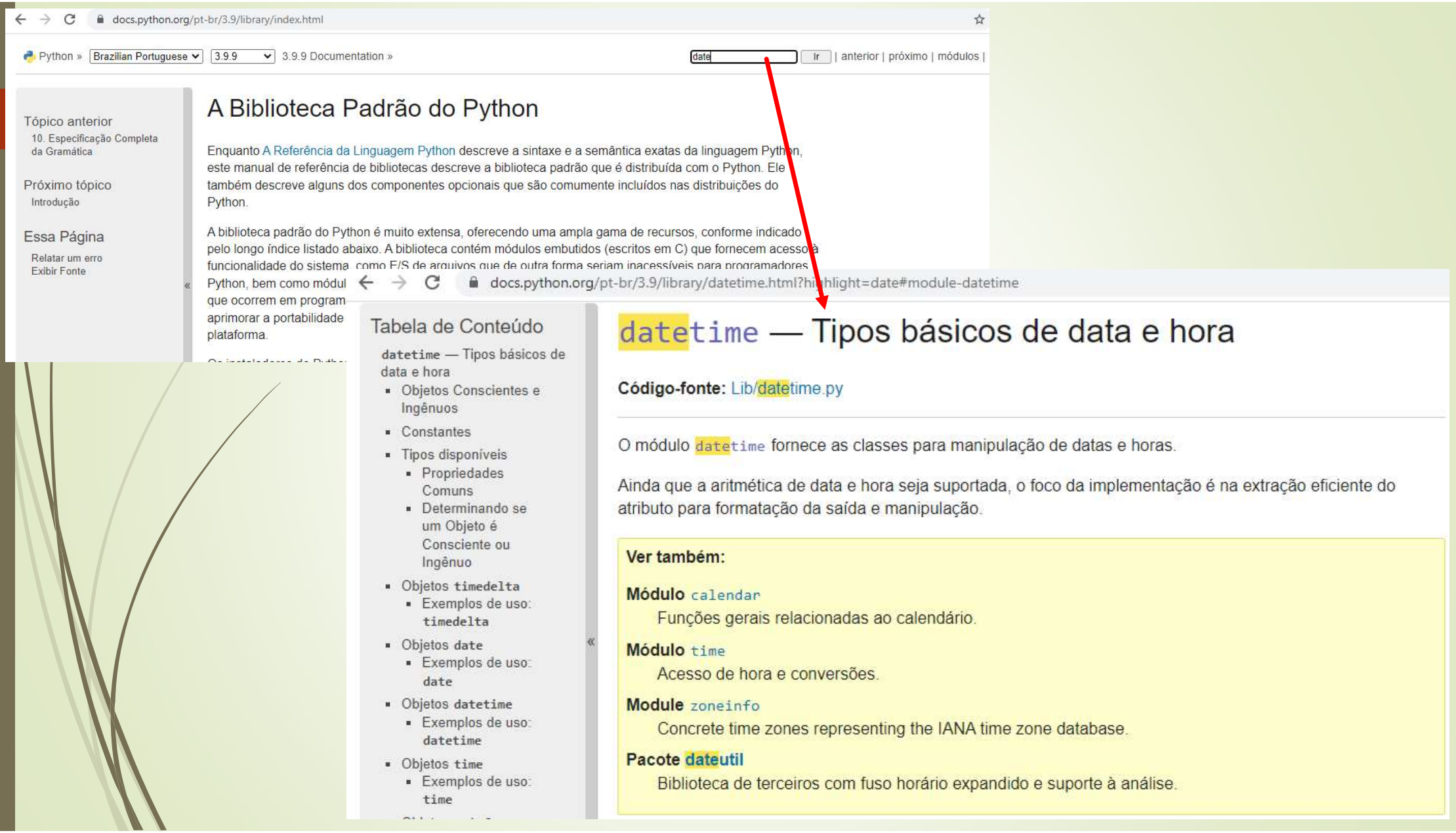
Importando módulos

- A **Biblioteca** Padrão do **Python** é uma coleção de módulos de script acessíveis a um programa **Python** para simplificar o processo de programação e remover a necessidade de reescrever os comandos mais usados.
- Eles podem ser usados chamando-os / importando-os no início de um script:
  - import (nome do módulo)
    - Importa o pacote completo
  - from (nome do módulo) import (nome da função)
    - Importa apenas a função que será utilizada

Para conhecer as bibliotecas do Python acesse: https://www.python.org/doc/

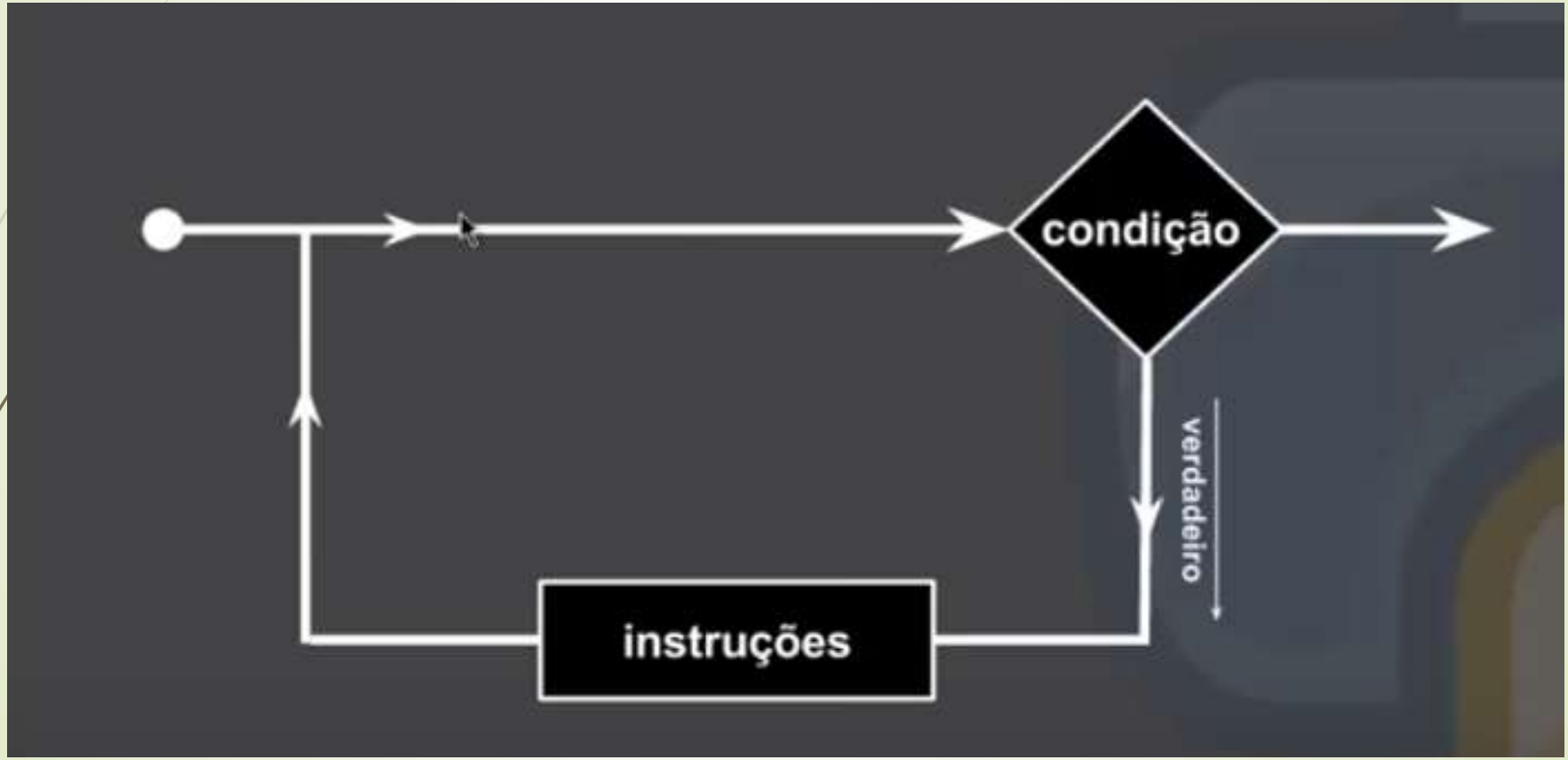





docs.python.org/pt-br/3.9/library/index.html
Python »
Brazilian Portuguese
3.9.9
3.9.9 Documentation »
date
Ir
| anterior | próximo | módulos |
Tópico anterior
10. Especificação Completa da Gramática
Próximo tópico
Introdução
Essa Página
Relatar um erro
Exibir Fonte
A Biblioteca Padrão do Python
Enquanto A Referência da Linguagem Python descreve a sintaxe e a semântica exatas da linguagem Python, este manual de referência de bibliotecas descreve a biblioteca padrão que é distribuída com o Python. Ele também descreve alguns dos componentes opcionais que são comumente incluídos nas distribuições do Python.
A biblioteca padrão do Python é muito extensa, oferecendo uma ampla gama de recursos, conforme indicado pelo longo índice listado abaixo. A biblioteca contém módulos embutidos (escritos em C) que fornecem acesso à funcionalidade do sistema, como E/S de arquivos que de outra forma seriam inacessíveis para programadores Python, bem como módul
que ocorrem em program
aprimorar a portabilidade
plataforma.
docs.python.org/pt-br/3.9/library/datetime.html?highlight=date#module-datetime
Tabela de Conteúdo
datetime — Tipos básicos de data e hora
Objetos Conscientes e Ingênuos
Constantes
Tipos disponíveis
Propriedades Comuns
Determinando se um Objeto é Consciente ou Ingênuo
Objetos timedelta
Exemplos de uso: timedelta
Objetos date
Exemplos de uso: date
Objetos datetime
Exemplos de uso: datetime
Objetos time
Exemplos de uso: time
datetime — Tipos básicos de data e hora
Código-fonte: Lib/datetime.py
O módulo datetime fornece as classes para manipulação de datas e horas.
Ainda que a aritmética de data e hora seja suportada, o foco da implementação é na extração eficiente do atributo para formatação da saída e manipulação.
Ver também:
Módulo calendar
Funções gerais relacionadas ao calendário.
Módulo time
Acesso de hora e conversões.
Module zoneinfo
Concrete time zones representing the IANA time zone database.
Pacote dateutil
Biblioteca de terceiros com fuso horário expandido e suporte à análise.

Estrutura de Repetição while

while



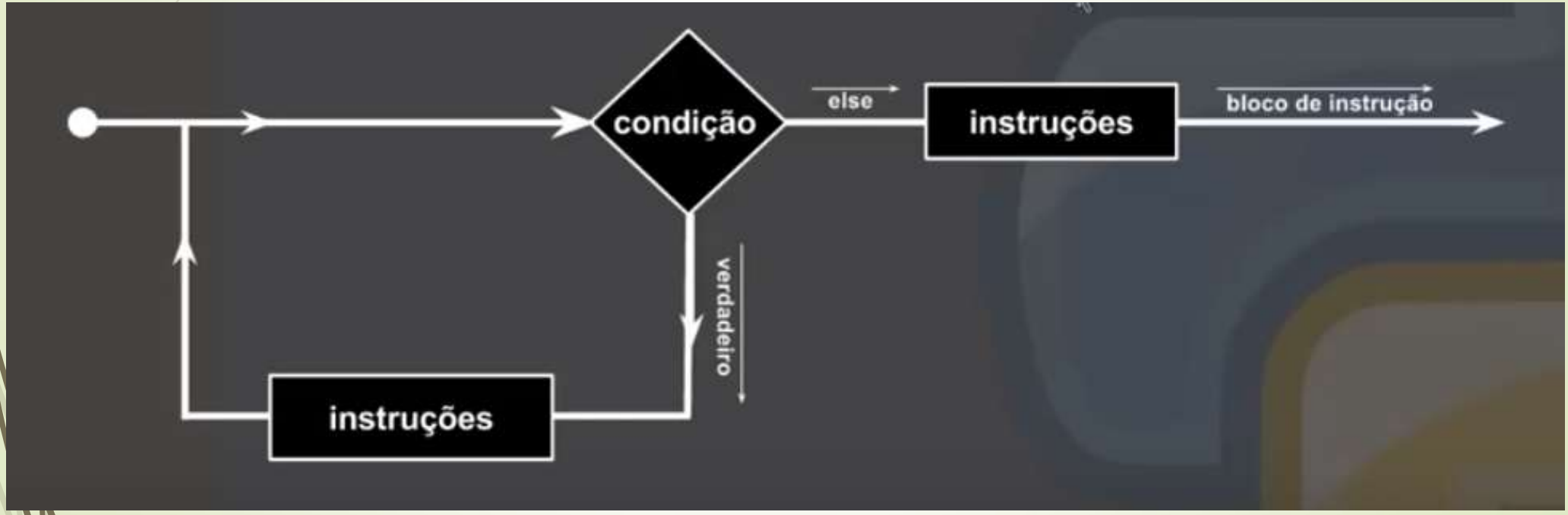
```python
'''jogo de adivinhar um número'''
from random import randrange
num = randrange(11)
cont = 1
teclado = int(input('Digite um número entre 0 e 10:'))
while (num != teclado):
    teclado = int(input('Digite novamente!'))
    cont += 1
print('Parabéns! Você acertou! O número secreto é: {}.\n Você fez {} tentativas'
      .format(num, cont))

```

Exemplo de uso do comando ‘while’

```
Digite um número entre 0 e 10:1
Digite novamente!2
Digite novamente!3
Digite novamente!4
Digite novamente!5
Digite novamente!6
Parabéns! Você acertou! O número secreto é: 6.
 Você fez 6 tentativas
```

While-else


condição
else
instruções
bloco de instrução
verdadeiro
instruções

```python
x=0
while x < 5:
    print(x, end=' ')
    x += 1
else:
    print('\nValor de x maior ou igual a 5. Entrou no else')

```

```
0 1 2 3 4
Valor de x maior ou igual a 5. Entrou no else
```

instrução break

- A instrução **break** finaliza a iteração e o código continua sua execução normalmente. O objetivo dessa instrução é fornecer a capacidade de forçar a interrupção da iteração.

- Em **Python**, a instrução **break** oferece a possibilidade de sair de um loop quando uma condição externa é acionada. A instrução **break** será colocada dentro do bloco de código abaixo da sua instrução de loop, geralmente após uma instrução condicional if

```
'''
# dará erro porque valor de n ainda não é conhecido
while n != 999:
    n = int(input('Digite um número:'))
'''
#Fazer um programa que leia números inteiros da tela e os some
# até que o número  999 seja digitada e encerre o programa
s = 0
while True:
    n = int(input('Digite um número:'))
    if n == 999:
        break
    s += n
print('A soma vale {}'.format(s))
```

Exemplo de uso do comando 'break':

```
Digite um número:5
Digite um número:4
Digite um número:3
Digite um número:999
A soma vale 12
```

usando o while para controlar a continuidade das execuções...

```
1  r = 'S'
2  while r == 'S':
3      n = int(input('Digite um valor: '))
4      r = str(input('Quer continuar? [S/N] ')).upper()
5  print('Fim')
```

```
Digite um valor: 3
Quer continuar? [S/N] s
Digite um valor: 6
Quer continuar? [S/N] s
Digite um valor: 8
Quer continuar? [S/N] n
Fim
```

Exemplo

- Faça um programa que leia um número qualquer e calcule seu fatorial
  - 4.1) usando o laço for
  - 4.2) usando o laço while
  - Ex: 3! 3 x 2 x 1 = 6

```python
from math import factorial
n = int(input('Digite um número:'))
f = factorial(n)
print(f'O fatorial de {n}! é {f}.')

# versão while
n1 = int(input('Digite um número para calcular seu fatorial:'))
c = n1
f = 1
print('{}! = '.format(n1), end='')
while c > 0:
    print('{}'.format(c), end='')
    print(' x ' if c > 1 else ' = ', end='')
    f *= c
    c -= 1
print('{}.'.format(f))
```

```python
# versão for
f1 = n2 = int(input('Informe um número (versão for): '))
for c2 in range(n2 - 1, 1, -1):
    f1 *= c2
print(f'O fatorial de {n2} é: {f1}')
```